

Proyecto 2

CS2023 – Algoritmos y Estructuras de Datos

Departamento de Computer Science

1. Contexto

Durante el curso se han estudiado estructuras fundamentales como listas, pilas, colas, tablas hash, árboles binarios de búsqueda, heaps, árboles balanceados, backtracking y grafos. En este proyecto final, cada grupo deberá investigar, implementar y aplicar una estructura de datos avanzada no desarrollada directamente en clase, mostrando su utilidad mediante una aplicación o prototipo visual.

El objetivo principal no es únicamente programar una estructura de datos, sino demostrar que esta permite resolver un problema concreto de forma más eficiente que una solución ingenua. Para ello, cada grupo deberá implementar la estructura asignada desde cero, integrarla en una aplicación concreta y realizar una comparación empírica contra una alternativa básica.

2. Objetivos del proyecto

Al finalizar el proyecto, el grupo deberá ser capaz de:

- implementar correctamente una estructura de datos avanzada;
- explicar sus invariantes, operaciones y complejidad;
- aplicar la estructura a un problema concreto usando datos sintéticos, reales o semirreales, según corresponda a la aplicación;
- construir una aplicación o prototipo visual que permita observar el uso de la estructura;
- comparar experimentalmente la solución implementada contra una solución ingenua;
- comunicar los resultados mediante un reporte, una demostración y una exposición oral.

3. Modalidad de trabajo

El proyecto se realizará en grupos. A cada grupo se le asignará una de las siguientes estructuras de datos:

1. QuadTree.
2. R-Tree.
3. Red-Black Tree aumentado.

4. Suffix Tree.

La estructura asignada a cada grupo será determinada por sorteo. Cada grupo deberá desarrollar la opción que le corresponda, cumpliendo los requisitos mínimos definidos para dicha estructura y aplicación.

El grupo puede proponer variantes, pero estas deberán mantener el mismo nivel de complejidad algorítmica y deberán ser aprobadas por el docente.

4. Reglas generales

- La estructura de datos asignada debe ser implementada desde cero.
- No se permite usar implementaciones ya existentes de la estructura asignada.
- No se permite reemplazar la estructura asignada por contenedores estándar como `std::map`, `std::set`, `std::unordered_map` u otras estructuras equivalentes.
- Sí se permite usar estructuras auxiliares básicas, siempre que no sustituyan el núcleo del proyecto.
- En la opción de Red-Black Tree, se permite y se recomienda combinar el árbol con una tabla principal de registros, una Hash Table auxiliar por `id` y un Hash Set por nodo para manejar claves repetidas. Sin embargo, el Red-Black Tree aumentado debe seguir siendo el índice ordenado principal y debe estar implementado desde cero.
- Sí se permite usar librerías para visualización, interfaz gráfica, lectura de archivos o manejo de ventanas.
- La aplicación debe permitir comparar la estructura implementada contra una solución ingenua.
- La aplicación debe ser ejecutable localmente y debe incluir instrucciones claras de compilación y ejecución.
- Si se usa un dataset externo, el grupo debe citar la fuente, el enlace y la licencia o condiciones de uso cuando corresponda.

5. Alcance de la visualización

La visualización no debe interpretarse como una obligación de animar paso a paso todas las operaciones internas de la estructura. El objetivo principal es que la aplicación permita observar el uso de la estructura en una consulta concreta y comparar su comportamiento contra una solución ingenua.

Como mínimo, cada aplicación debe visualizar:

- los datos de entrada;
- la consulta u operación realizada;
- los resultados obtenidos;

- alguna evidencia del recorrido, filtrado o acceso realizado por la estructura, como nodos visitados, regiones exploradas, MBRs relevantes, subdivisiones activas, rutas de búsqueda, fragmentos encontrados o claves recorridas;
- las métricas principales de comparación contra la solución ingenua.

La visualización interna detallada de operaciones como splits, rotaciones, recoloreos o fases de construcción será considerada una mejora, pero no un requisito obligatorio.

6. Comparación experimental obligatoria

Todos los grupos deben comparar su estructura contra una solución ingenua. La comparación debe realizarse con diferentes tamaños de entrada para evidenciar cómo cambia el rendimiento al aumentar la cantidad de datos.

Cada grupo debe reportar, como mínimo, experimentos con tres tamaños de entrada:

- tamaño pequeño;
- tamaño mediano;
- tamaño grande.

Los valores concretos dependerán de la opción asignada y de la capacidad de la máquina utilizada. El grupo debe justificar los tamaños elegidos.

La comparación debe incluir al menos:

- tiempo de construcción de la estructura;
- tiempo promedio de consulta;
- cantidad de elementos revisados, comparaciones realizadas o nodos visitados;
- resultados para diferentes tamaños de entrada;
- una breve interpretación de los resultados.

Estructura	Solución ingenua	Operación a comparar
QuadTree	comparar todos contra todos	vecinos o colisiones
R-Tree	búsqueda lineal	KNN o rango rectangular
Red-Black Tree	escaneo lineal de tabla	igualdad, rango, ranking, percentiles
Suffix Tree	búsqueda directa en texto	búsqueda de patrones

7. Tecnologías permitidas

El proyecto puede desarrollarse en C++ o en otro lenguaje autorizado por el docente. Para la parte visual se permite usar, entre otras opciones:

- SFML;

- raylib;
- SDL;
- Qt;
- OpenGL básico;
- ImGui;
- una interfaz web, siempre que el núcleo algorítmico sea implementado por el grupo;
- visualización por consola o generación de archivos de salida, siempre que sea clara y permita entender las consultas realizadas.

8. Opciones de proyecto

8.1. Opción A: QuadTree – Simulador de partículas 2D

Aplicación concreta

El grupo deberá construir un simulador 2D de partículas u objetos circulares en movimiento. Cada partícula tendrá posición, velocidad y radio. En cada frame, la aplicación deberá actualizar las posiciones, reconstruir o actualizar el QuadTree y usarlo para detectar vecinos cercanos o posibles colisiones.

La aplicación debe permitir generar datos sintéticos configurables. No es obligatorio usar un dataset externo.

Modelo de objeto sugerido

```
struct Particle {
    int id;
    double x, y;
    double vx, vy;
    double radius;
};
```

Datos de entrada

La aplicación debe permitir configurar:

- número de objetos n ;
- tamaño del espacio 2D;
- capacidad máxima por nodo del QuadTree;
- radios de las partículas;
- velocidades;
- distribución inicial de los objetos.

Como mínimo, se deben implementar tres escenarios de distribución:

- distribución uniforme;
- distribución con clusters;
- distribución con una zona de alta densidad.

Funcionalidades mínimas

La aplicación debe permitir:

- insertar objetos en el QuadTree;
- subdividir regiones cuando se supere una capacidad máxima;
- actualizar posiciones de objetos frame por frame;
- reconstruir o actualizar el QuadTree después del movimiento;
- consultar objetos dentro de una región rectangular;
- consultar objetos cercanos a un punto;
- detectar posibles colisiones o encuentros entre objetos;
- comparar contra una solución de fuerza bruta.

Qué se debe visualizar

La visualización debe enfocarse en la simulación y en los resultados de las consultas. Como mínimo, debe mostrar:

- las partículas u objetos en el plano;
- las subdivisiones actuales del QuadTree;
- la región consultada o el radio de vecindad;
- los objetos candidatos retornados por el QuadTree;
- las colisiones o vecinos detectados;
- el número de comparaciones realizadas por fuerza bruta y por QuadTree;
- el tiempo por frame o por consulta.

No es obligatorio animar cada inserción individual en el QuadTree.

Experimentos requeridos

El grupo debe comparar QuadTree contra fuerza bruta usando al menos tres tamaños de simulación. Se sugieren:

- 1,000 objetos;
- 5,000 objetos;
- 10,000 objetos o más, si la máquina lo permite.

Para cada tamaño, se debe reportar:

- tiempo promedio por frame;
- número promedio de comparaciones;
- número de candidatos revisados por objeto;
- efecto de la distribución espacial en el rendimiento.

8.2. Opción B: R-Tree – Buscador espacial con consultas por rango y KNN

Aplicación concreta

El grupo deberá construir un buscador espacial sobre un conjunto de puntos de interés. Los objetos pueden representar ciudades, restaurantes, hospitales, universidades, parques u otros puntos geográficos. La aplicación debe permitir seleccionar una región rectangular o un punto de consulta, y responder usando un R-Tree.

La implementación base deberá corresponder a un **R-Tree clásico con split cuadrático**. No se exige implementar R-Tree, reinserción forzada ni optimizaciones avanzadas.

Datos de entrada

Se recomienda usar **GeoNames Cities**, especialmente las versiones `cities1000`, `cities5000` o `cities15000`, ya que son archivos pequeños, fáciles de parsear y con coordenadas geográficas reales.

Como alternativa, se puede usar un subconjunto de **OpenStreetMap** filtrado para Lima o Perú, extrayendo únicamente puntos de interés.

Cada objeto puede modelarse como:

```
struct SpatialObject {  
    int id;  
    std::string name;  
    double x, y;  
    std::string category;  
};
```

Funcionalidades mínimas

La aplicación debe permitir:

- cargar objetos espaciales desde un archivo;
- insertar objetos espaciales;
- eliminar objetos espaciales;
- construir el R-Tree a partir de los datos;
- implementar split cuadrático como estrategia obligatoria de división de nodos;
- actualizar correctamente los MBRs después de inserciones, eliminaciones y divisiones;
- realizar consultas por rango rectangular;
- realizar consultas de k vecinos más cercanos;
- comparar contra una búsqueda lineal.

Requisito sobre split cuadrático

Cuando un nodo exceda su capacidad máxima, el grupo deberá aplicar split cuadrático:

1. elegir dos entradas semilla que produzcan el mayor desperdicio de área si se colocaran juntas;
2. crear dos grupos iniciales a partir de dichas semillas;
3. asignar las entradas restantes al grupo donde produzcan menor incremento de área;
4. respetar el mínimo de entradas por nodo cuando corresponda;
5. recalcular los MBRs de los nodos resultantes.

El reporte deberá incluir una explicación del split cuadrático. No es obligatorio animar el split paso a paso en la aplicación.

Qué se debe visualizar

La visualización debe enfocarse en el mapa, las consultas y los resultados. Como mínimo, debe mostrar:

- los puntos u objetos espaciales;
- la región rectangular consultada;
- el punto de consulta para KNN;
- los resultados encontrados;
- los MBRs relevantes del R-Tree;
- los nodos o regiones visitadas durante la consulta;

- el tiempo de consulta con R-Tree y con búsqueda lineal.

No es obligatorio dibujar todo el árbol como una jerarquía ni animar cada inserción o split. Basta con mostrar los MBRs relevantes y la evidencia de las regiones exploradas.

Experimentos requeridos

El grupo debe comparar R-Tree contra búsqueda lineal usando al menos tres tamaños de dataset. Se sugieren:

- 1,000 objetos;
- 5,000 objetos;
- 15,000 objetos o más, si el dataset y la máquina lo permiten.

Para cada tamaño, se debe reportar:

- tiempo de construcción del R-Tree;
- tiempo promedio de consulta por rango;
- tiempo promedio de consulta KNN;
- cantidad de nodos visitados por el R-Tree;
- cantidad de elementos revisados por búsqueda lineal;
- efecto del tamaño de la consulta rectangular o del valor de k .

8.3. Opción C: Red-Black Tree aumentado – Mini base de datos en RAM con índices ordenados

Aplicación concreta

El grupo deberá construir una mini base de datos en memoria. La aplicación debe cargar o generar registros, permitir consultas por `id` y consultas ordenadas sobre una columna numérica o comparable.

La base de datos debe separar claramente:

1. una tabla principal de registros;
2. una Hash Table por `id`;
3. un índice Red-Black Tree aumentado sobre una columna como `score`, `price`, `age`, `rating`, `quantity` o `timestamp`.

El Red-Black Tree no debe verse únicamente como un diccionario `clave -> valor`, sino como un índice ordenado aumentado capaz de responder consultas de rango, ranking, selección y percentiles.

Datos de entrada

Se recomienda usar uno de los siguientes datasets:

- **MovieLens small o MovieLens 100K**: ratings de películas, usuarios, timestamps y películas;
- **UCI Online Retail II**: transacciones de una tienda online con precios, cantidades, fechas, clientes y países.

También se permite usar un dataset sintético de estudiantes, productos o transacciones, siempre que tenga suficientes registros para la comparación experimental.

Modelo de registro sugerido

```
struct Record {  
    int id;  
    std::string name;  
    int age;  
    double score;  
    std::string category;  
};
```

La tabla principal puede almacenarse en un arreglo, vector o estructura propia. La Hash Table debe mapear `id ->Record` o `id ->posicion`, y el índice Red-Black Tree debe mapear `key ->coleccion de ids`.

```
HashTable<int, Record> recordsById;  
RedBlackTree<double, HashSet<int>> scoreIndex;
```

Nodo aumentado

Cada nodo del árbol debe almacenar, como mínimo:

```
struct Node {  
    Key key;  
    HashSet<int> rowIds; // o vector<int> si se justifica  
  
    Color color;  
    Node* left;  
    Node* right;  
    Node* parent;  
  
    int subtreeSize;  
};
```

El campo `subtreeSize` debe contar la cantidad de registros en el subárbol, no solamente la cantidad de nodos. Por tanto:

$$\text{subtreeSize} = \text{size}(\text{left}) + \text{rowIds.size}() + \text{size}(\text{right}).$$

Este campo debe actualizarse después de inserciones, eliminaciones y rotaciones.

Se recomienda que `rowIds` sea un Hash Set implementado por el grupo para que insertar y eliminar un `id` asociado a una clave repetida sea eficiente en promedio. Si el grupo usa un `vector<int>`, deberá explicar el costo adicional de eliminar un `id` dentro del vector cuando existen muchas claves repetidas.

Funcionalidades mínimas

La Hash Table debe permitir, como mínimo:

```
insert(id, record)
remove(id)
contains(id)
get(id)
update(id, record)
```

El Red-Black Tree aumentado debe permitir, como mínimo:

```
insert(key, rowId)
remove(key, rowId)
search(key)
rangeQuery(l, r)
countRange(l, r)
rank(key)
select(k)
```

A nivel de aplicación, se debe permitir:

```
insertRecord(record)
deleteRecord(id)
updateIndexedField(id, newValue)
findById(id)
findEqual(field, value)
findBetween(field, l, r)
countBetween(field, l, r)
topK(field, k)
median(field)
percentile(field, p)
```

No es obligatorio implementar SQL completo. Puede usarse una interfaz con comandos predefinidos, botones o formularios.

Coordinación entre estructuras

Las operaciones de la aplicación deben mantener consistentes la tabla principal, la Hash Table y el índice Red-Black Tree.

```
insertRecord(record):
    recordsById.insert(record.id, record)
    scoreIndex.insert(record.score, record.id)

updateIndexedField(id, newScore):
    record = recordsById.get(id)
    scoreIndex.remove(record.score, id)
    record.score = newScore
    recordsById.update(id, record)
    scoreIndex.insert(newScore, id)
```

```
deleteRecord(id):
    record = recordsById.get(id)
    scoreIndex.remove(record.score, id)
    recordsById.remove(id)
```

Qué se debe visualizar

La visualización debe enfocarse en las consultas de la mini base de datos y en el uso del índice. Como mínimo, debe mostrar:

- una tabla o listado de registros;
- el resultado de una búsqueda por `id`;
- el resultado de consultas por igualdad, rango, top- k , mediana o percentil;
- el árbol Red-Black por niveles o una representación equivalente;
- color de los nodos;
- valor de `subtreeSize`;
- claves o nodos visitados durante una consulta;
- comparación contra escaneo lineal de la tabla.

No es obligatorio animar cada rotación o recoloreo. Basta con mostrar el estado del árbol después de operaciones relevantes y evidenciar qué nodos se recorrieron.

Complejidad esperada

Operación	Complejidad esperada
Buscar por <code>id</code> usando Hash Table	$O(1)$ promedio
Insertar registro	$O(1)$ promedio $+O(\log u)$
Eliminar registro usando Hash Set en el nodo	$O(1)$ promedio $+O(\log u)$
Eliminar registro usando vector en el nodo	$O(1)$ promedio $+O(\log u + t)$
Actualizar campo indexado	$O(1)$ promedio $+O(\log u)$
Buscar valor exacto en índice	$O(\log u + k)$
Buscar rango en índice	$O(\log u + k)$
Contar elementos en rango	$O(\log u)$
Obtener el k -ésimo elemento	$O(\log u)$
Mediana o percentil	$O(\log u)$
Escaneo lineal de tabla	$O(n)$

Aquí n representa la cantidad total de registros, u la cantidad de claves distintas en el Red-Black Tree, k la cantidad de registros devueltos por una consulta y t la cantidad de registros asociados a una misma clave.

Experimentos requeridos

El grupo debe comparar el índice Red-Black Tree contra escaneo lineal usando al menos tres tamaños de tabla. Se sugieren:

- 10,000 registros;
- 100,000 registros;
- 500,000 registros o más, si la máquina lo permite.

Para cada tamaño, se debe reportar:

- tiempo de carga o inserción;
- tiempo de búsqueda por id;
- tiempo de consulta por igualdad en el campo indexado;
- tiempo de consulta por rango;
- tiempo para mediana o percentil;
- número de nodos visitados por el índice;
- número de registros revisados por escaneo lineal.

8.4. Opción D: Suffix Tree – Buscador indexado de documentos

Aplicación concreta

El grupo deberá construir un buscador indexado de documentos. La aplicación permitirá cargar un PDF con texto seleccionable o un archivo de texto, extraer o leer su contenido, construir un Suffix Tree y realizar búsquedas eficientes de palabras o frases.

La funcionalidad será similar a un *Ctrl + F*, pero con una diferencia importante: antes de buscar, el sistema construirá un índice sobre el texto del documento. Luego, cada consulta se resolverá recorriendo el Suffix Tree.

La construcción del Suffix Tree deberá realizarse mediante el **algoritmo de Ukkonen**. El grupo deberá explicar el proceso de construcción incremental, los suffix links, las fases, las extensiones, el active point, el remainder y el rol del carácter terminal.

Pipeline esperado

PDF o TXT
-> extraccion o lectura de texto
-> normalizacion
-> construccion del Suffix Tree con Ukkonen
-> busqueda de patron
-> ocurrencias encontradas
-> resaltado o listado de fragmentos

El árbol no se construye sobre el archivo PDF binario, sino sobre el texto extraído. La aplicación debe funcionar con PDFs que contengan texto seleccionable o con archivos `.txt`. Los PDFs escaneados como imagen requerirían OCR y serán considerados funcionalidad adicional.

Funcionalidades mínimas

El Suffix Tree debe implementarse usando Ukkonen y debe exponer, como mínimo:

```
buildUkkonen(text)
contains(pattern)
countOccurrences(pattern)
findOccurrences(pattern)
```

Además, la implementación debe manejar explícitamente:

- carácter terminal único al final del texto;
- suffix links;
- active point;
- active edge, active length y active node;
- remainder o número de sufijos pendientes;
- hojas con índices de sufijo para recuperar ocurrencias.

La aplicación debe permitir:

- cargar un PDF con texto seleccionable o un archivo `.txt`;
- extraer o leer el texto;
- normalizar el texto;
- construir el Suffix Tree;
- buscar una palabra o frase;
- contar ocurrencias;
- listar posiciones o páginas donde aparece el patrón;
- mostrar fragmentos donde aparece la coincidencia;
- comparar contra una búsqueda ingenua.

Qué se debe visualizar

La visualización debe enfocarse en la búsqueda y sus resultados. Como mínimo, debe mostrar:

- el texto o fragmentos del documento;
- una barra o entrada de búsqueda;
- el patrón buscado;
- ocurrencias resaltadas o listadas;
- número total de coincidencias;

- posiciones o páginas donde aparece el patrón;
- la ruta recorrida en el Suffix Tree para el patrón;
- tiempo de construcción y tiempo de búsqueda;
- comparación contra búsqueda ingenua.

No es obligatorio dibujar todo el Suffix Tree, ya que puede ser demasiado grande. Es suficiente mostrar la ruta del patrón, el nodo o subárbol asociado y las ocurrencias recuperadas.

Complejidad esperada

Si el texto tiene longitud n , el patrón tiene longitud m y existen k ocurrencias, se espera el siguiente análisis:

Operación	Complejidad esperada
Construcción con Ukkonen	$O(n)$
Búsqueda de patrón	$O(m + k)$
Búsqueda ingenua	$O(nm)$ o $O(n)$, según implementación
Recuperación de ocurrencias	$O(k)$ después de encontrar el patrón

La construcción ingenua en $O(n^2)$ puede ser usada únicamente como herramienta de validación o comparación, pero no reemplaza el requisito de implementar Ukkonen.

Datos sugeridos

Se recomienda usar libros o documentos con texto seleccionable. Algunas opciones razonables son:

- libros de **OpenStax** en PDF;
- papers académicos en PDF;
- documentos del curso;
- archivos `.txt` suficientemente largos;
- una colección pequeña de documentos limpios proporcionada por el grupo.

Por consumo de memoria, se recomienda indexar un documento a la vez y limitar el texto a tamaños configurables:

- 100,000 caracteres para una demo básica;
- 500,000 caracteres para una demo intermedia;
- 1,000,000 caracteres como prueba avanzada, si la máquina lo permite.

Experimentos requeridos

El grupo debe comparar Suffix Tree contra búsqueda ingenua usando al menos tres tamaños de texto. Se sugieren:

- 100,000 caracteres;
- 500,000 caracteres;
- 1,000,000 caracteres, si la máquina lo permite.

Para cada tamaño, se debe reportar:

- tiempo de construcción del Suffix Tree;
- tiempo promedio de búsqueda usando Suffix Tree;
- tiempo promedio de búsqueda ingenua;
- número de ocurrencias encontradas;
- longitud de los patrones utilizados;
- efecto del tamaño del texto y del patrón en el rendimiento.

9. Fuentes sugeridas de datasets

Los siguientes datasets son de acceso gratuito. Si se utiliza alguno de ellos, el grupo deberá citar la fuente en el README, reporte y presentación.

Opción	Dataset sugerido	Enlace
R-Tree	GeoNames Cities	https://download.geonames.org/export/dump/
R-Tree	OpenStreetMap Perú – Geofabrik	https://download.geofabrik.de/south-america/peru.html
Red-Black Tree	MovieLens datasets	https://grouplens.org/datasets/movielens/
Red-Black Tree	UCI Online Retail II	https://archive.ics.uci.edu/dataset/502/online+retail+ii
Suffix Tree	OpenStax textbooks	https://openstax.org/

Para QuadTree, el uso de dataset externo no es obligatorio, ya que la aplicación debe generar datos sintéticos configurables. Si el grupo desea usar datos reales como extensión, puede emplear puntos GPS, trayectorias o coordenadas espaciales, citando la fuente correspondiente.

10. Consideraciones finales

El proyecto debe evidenciar que el grupo comprende tanto la estructura de datos como su utilidad práctica. Una buena entrega no solo debe funcionar, sino también permitir observar por qué la estructura mejora una solución ingenua.

Se valorará especialmente que la aplicación permita visualizar el comportamiento relevante de la estructura para la consulta realizada: subdivisiones, nodos visitados, MBRs relevantes, claves recorridas, rutas de búsqueda, fragmentos encontrados o cualquier otro elemento que ayude a entender el algoritmo.

No se exige una interfaz gráfica compleja ni animaciones completas de todas las operaciones internas. La prioridad del proyecto es la implementación correcta de la estructura, su integración en una aplicación concreta y la comparación experimental contra una solución ingenua usando distintos tamaños de entrada.